Project Proposal & Literature Review

Lightweight Learned Image Compression for Laptop-Class Training

Mohamad Sherif Mohamed Abdelazeem Zeyad Tarek

Milestone 1
CSE681 — Digital Image Processing

Abstract

This project investigates learned image compression under limited computational resources, with the goal of reproducing a compact hyperprior-based codec that can be trained within the project period on laptop-class hardware. The model will use reduced channel widths and latent dimensions, and its rate–distortion performance will be evaluated using peak signal-to-noise ratio (PSNR) and multi-scale structural similarity (MS-SSIM) as functions of bits per pixel (BPP). The evaluation will compare the learned codec with a conventional JPEG baseline and published ELIC operating points, while examining how reducing model width and latent channel count affects reconstruction quality and computational cost. Training will use random 256×256 crops from DIV2K, while final image-quality evaluation will use the Kodak dataset and, subject to availability and time, Tecnick.

1 Problem Statement and Motivation

Image compression aims to represent visual data using fewer bits while preserving the information and perceptual quality needed for reconstruction. Conventional codecs such as JPEG provide practical and widely used solutions, but learned image compression offers an alternative in which an encoder, a probability model, and a decoder are optimized jointly from data. In particular, hyperprior-based methods estimate the statistical properties of latent representations and use this information to support more efficient entropy coding (Ballé et al., 2018).

Despite their promising rate–distortion performance, learned codecs can require substantial computational resources and model capacity. Reproducing a large architecture may therefore be impractical within a short academic project or on laptop-class hardware. The central problem addressed in this work is whether a compact, reduced-width hyperprior codec can be trained and evaluated within the available time and compute budget, while retaining useful compression performance.

The project has four connected objectives. First, it will reproduce a compact hyperprior-based learned codec, reducing the channel width and latent channel count to make training more manageable. Second, it will measure rate–distortion performance using PSNR and MS-SSIM against BPP, and compare the results with a JPEG baseline and published ELIC operating points. Third, it will investigate the trade-off between reconstruction quality and model complexity as the network width and latent dimensionality are reduced. Finally, it will document practical considerations such as training feasibility, model configuration, and decoding cost.

The experimental design is intended to make the comparison meaningful rather than simply compare nominal quality settings. DIV2K training and validation images will provide the training source, with random 256×256 crops used during training. The Kodak dataset, containing 24 images, will serve as the primary evaluation benchmark; Tecnick may be added if time and dataset

availability permit. JPEG will provide a conventional baseline, while ELIC will provide a reference to a more recent efficient learned-compression approach. Comparisons will be based on measured BPP and image-quality metrics, with care taken to distinguish reproduced experiments from operating points reported in prior publications.

This work connects directly to digital image processing through transform-based representation, quantization, entropy coding, reconstruction, and objective image-quality assessment. Its motivation is not only to obtain a competitive rate–distortion curve, but also to understand the engineering trade-offs that arise when a learned compression architecture is scaled down for limited hardware. A successful outcome will establish a reproducible evaluation framework and clarify how much model capacity can be reduced before compression quality or practical usability is significantly affected.

2 Related Work

Learned image compression replaces the hand-designed transforms of codecs such as JPEG (Wallace, 1992) with neural networks trained end to end on a rate–distortion objective. Ballé et al. (2017) introduced the basic recipe: a convolutional analysis transform maps the image to a latent y, which is quantized and entropy coded, and a synthesis transform reconstructs the image. Quantization is replaced by additive uniform noise during training so that the rate term remains differentiable.

Ballé et al. (2018) added a *scale hyperprior*: a second, much smaller latent z is transmitted as side information and predicts the standard deviation of a Gaussian for every element of y. This captures spatially varying statistics that a fixed factorized prior misses and is the architecture we reproduce at reduced width. Minnen et al. (2018) combined the hyperprior with a spatial autoregressive context model. This improves rate–distortion performance, but decoding becomes sequential and therefore slow, which conflicts with our laptop-class goal.

ELIC (He et al., 2022) addresses this cost with unevenly grouped channel contexts and a checkerboard spatial context. It keeps most of the gain of context modelling while decoding in parallel, and its published operating points are our stretch target. More recent work increases capacity in different ways. Liu et al. (2023) mix Transformer and CNN blocks in the transforms and entropy model, gaining rate–distortion performance at a higher computational cost. Duan et al. (2023) generalize the two-level hyperprior to a deeper hierarchy of quantized latent variables. Both show that quality grows with model complexity, which motivates our study of how far width and latent channels can shrink before performance drops. CompressAI (Bégaint et al., 2020) provides reference PyTorch implementations, pretrained models and evaluation tools for these codecs; we build on it to keep our results comparable with the literature.

3 Dataset and Baseline Plan

Training data. We train on DIV2K (Agustsson and Timofte, 2017), using both the training split (800 images) and the validation split (100 images) of the high-resolution set, about 2K pixels wide (`https://data.vision.ee.ethz.ch/cvl/DIV2K/`). Our loader, `DIV2KDataset`, draws a new random 256×256 crop from an image each time it is accessed and applies random horizontal and vertical flips. It returns RGB `float32` tensors in $[0, 1]$, so a batch has shape $[B, 3, 256, 256]$. Images are never resized, and an image smaller than the crop raises an error. Random cropping keeps memory and compute low enough for laptop training while still covering the full images over many epochs. A verification script checks that every image decodes before training starts; all 900 DIV2K images and all 24 Kodak images pass. The dataset location is configured in one

Table 1: Key references and their relevance to this project.

Reference	Year	Relevance
(Ballé et al., 2017)	2017	End-to-end learned transform coding; noise relaxation of quantization.
(Ballé et al., 2018)	2018	Scale-hyperprior codec; the architecture we reproduce at reduced width.
(Minnen et al., 2018)	2018	Autoregressive context plus hyperprior; better RD, but slow serial decoding.
(He et al., 2022)	2022	Efficient parallel context model; stretch-goal operating points.
(Liu et al., 2023)	2023	Transformer–CNN transforms; upper end of the complexity trade-off.
(Duan et al., 2023)	2023	Hierarchical quantized VAE generalizing the hyperprior.

place (`data/paths.py`), so every team member can keep the data wherever they like on their own machine.

Evaluation data. The Kodak image dataset (Eastman Kodak, n.d.) is used as the common evaluation dataset for the compression methods in this project. It contains 24 test images, each with a resolution of 768×512 pixels (`https://r0k.us/graphics/kodak/kodak/`). The same original images will be used to evaluate both the conventional JPEG baseline and the learned compression model. Kodak is never used for training. As a second, higher-resolution test set we plan to use Tecnick from the TESTIMAGES archive (Asuni and Giachetti, 2014) (`https://testimages.org/`), the 100 RGB images at 1200×1200 commonly used in the learned-compression literature. **[TODO: Confirm the Tecnick subset and its license after download (W5).]**

JPEG baseline. The baseline uses Pillow (libjpeg) at its default settings, including 4:2:0 chroma subsampling. We run quality factors $Q \in \{10, 20, 30, 50, 70, 85, 95\}$. Each full image is encoded and decoded; BPP uses the actual JPEG file size including headers, $\text{bpp} = 8 \cdot \text{bytes}/(W \cdot H)$. PSNR is measured over RGB channels with peak value 255, MS-SSIM (Wang et al., 2003) uses the shared evaluation functions, and decode time is recorded. Table 2 contains the exact means for our full quality list. Student C's preliminary plots show five operating points; the lowest-rate measurements agree at about 0.33 bpp and 26.7 dB.

Table 2: Pillow JPEG on Kodak (24 images, mean per quality factor).

Q	bpp	PSNR (dB)
10	0.327	26.67
20	0.508	29.14
30	0.660	30.49
50	0.906	32.17
70	1.239	33.92
85	1.857	36.46
95	3.392	40.56

The learned codec will be evaluated on the same Kodak images with the same metric definitions. Its final BPP will use the actual entropy-coded bitstream, not the training-time estimate; rate–distortion curves will be compared at matched BPP by interpolation.

4 Proposed Method

We plan to develop a reduced-width learned image-compression model based on the scale-hyperprior approach of Ballé et al. (2018). As shown in Figure 1, the encoder transforms an image into a latent representation. A hyperprior network produces additional information that helps estimate the latent distribution for entropy coding. The decoder uses the coded latents and hyperprior information to reconstruct the image.

To study the effects of model capacity and target bitrate, we will evaluate a small set of channel-width configurations and rate settings. For each selected configuration, the training loop will process a batch of RGB image crops. The encoder and hyperprior produce quantized latents, and the decoder reconstructs each crop. We will train the model by minimizing the rate–distortion objective in Eq. 1:

$$\mathcal{L} = R + \lambda D, \tag{1}$$

where R is the estimated bitrate in bits per pixel, D is the reconstruction distortion, and λ, hyperparameter which controls the trade-off between bitrate and image quality. We will use mean squared error (MSE), defined in Eq. 2, for the distortion term:

$$D = \frac{1}{3HW} \sum_{c=1}^{3} \sum_{u=1}^{H} \sum_{v=1}^{W} (x_{cuv} - \hat{x}_{cuv})^2 , \tag{2}$$

where x is the input crop, $\hat{x}$ is its reconstruction, and c indexes the three RGB channels. The optimizer will backpropagate this loss to update the model weights over repeated batches. We will record validation results and save model checkpoints during training. The exact widths, rate settings, and training parameters will be selected after a feasibility test. The likelihood-based rate is used for training; final reported BPP will be calculated from the actual encoded bitstream.

After training, each selected model will be evaluated on the same Kodak images as the JPEG baseline. We will encode and decode the images and compare BPP, PSNR, and MS-SSIM using the evaluation pipeline described below. Final BPP will use the actual encoded bitstream size.

Student A data pipeline. Training batches are drawn from `DIV2KDataset` with a shuffled PyTorch `DataLoader`; each batch has shape $[B, 3, 256, 256]$ and values in $[0, 1]$. A new crop and flip is sampled on each access. The model requires height and width divisible by 64: its analysis transform downsamples by 16 and its hyper-analysis transform by a further 4. Thus a 256×256 crop gives a 16×16 latent y and a 4×4 hyperlatent z. Kodak's 768×512 images can be evaluated uncropped. Tecnick's 1200×1200 images will be padded to 1216×1216 and cropped back after decoding; BPP and image quality will be calculated using the original pixel dimensions.

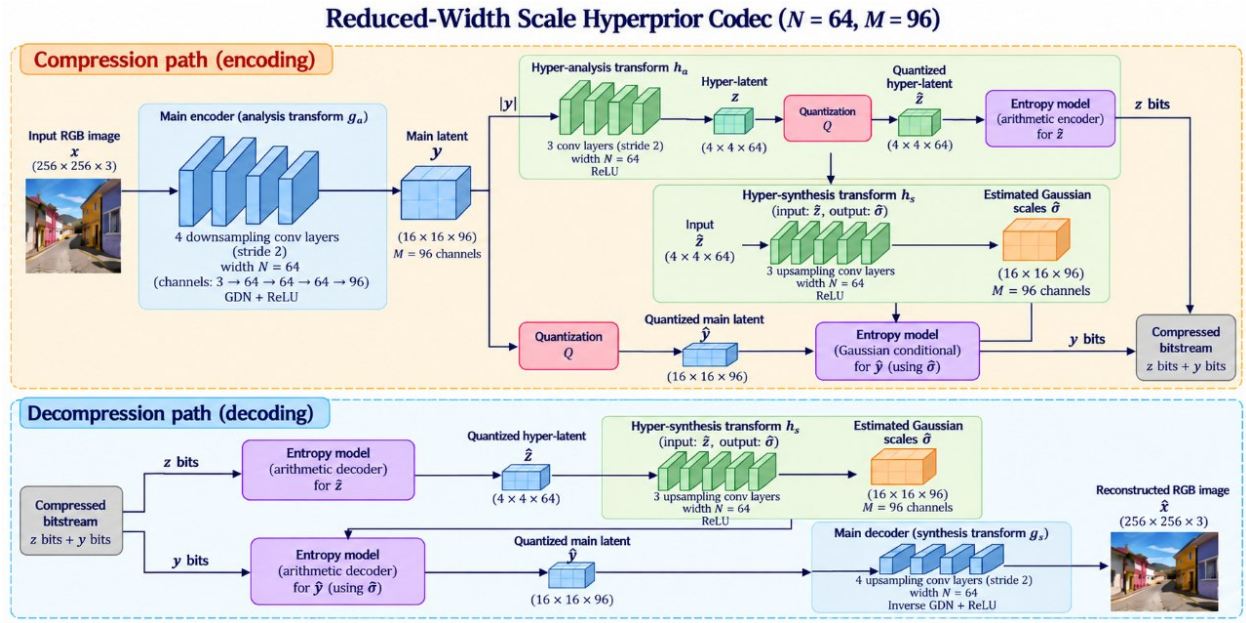


Figure 1: Planned architecture for the learned image-compression codec.

5 Evaluation Plan

The evaluation is designed to compare compression rate, reconstructed image quality, and decoding performance. Three primary image compression metrics are used.

- **Bits per pixel (BPP):** BPP measures the average number of encoded bits per image pixel. For a compressed file with size measured in bytes, BPP is calculated as shown in Eq. 3:

$$\text{BPP} = \frac{8 \times \text{compressed file size (bytes)}}{\text{image width} \times \text{image height}}. \tag{3}$$

  A lower BPP indicates a smaller encoded representation.

- **Peak signal-to-noise ratio (PSNR):** PSNR measures pixel-level reconstruction error and is reported in decibels (dB). For images represented with a maximum possible pixel value MAX, PSNR is calculated from the MSE as shown in Eq. 4:

$$\text{PSNR} = 10 \log_{10} \left(\frac{MAX^2}{\text{MSE}} \right). \tag{4}$$

  Higher PSNR generally indicates lower reconstruction error.

- **Multi-scale structural similarity (MS-SSIM):** MS-SSIM compares structural information at multiple image scales. Its commonly used range is 0 to 1, with values closer to 1 indicating greater structural similarity.

For each JPEG operating point, BPP, PSNR, and MS-SSIM are calculated for the Kodak images. The per-image measurements are retained, and the average metric values are used to produce

the summary plots. BPP is calculated from the actual compressed file size, rather than from an estimated entropy or target bitrate.

Two rate–distortion plots are generated:

- Average PSNR (dB) versus BPP.
- Average MS-SSIM versus BPP.

The same evaluation procedure will be applied to the learned codec. The comparison will use the same original Kodak images, image dimensions, color handling, and metric definitions. The final report will include the individual measurements where appropriate, averages, and rate–distortion curves. Decoding time will also be measured on the same hardware environment for both methods. Timing will use warm-up runs and repeated measurements; file reading time should be excluded where practical so that the measurement focuses on decoding.

5.1 Preliminary JPEG Baseline Results

The first evaluation experiment establishes the JPEG baseline on the Kodak dataset. The current plots show five operating points, with average BPP values spanning approximately 0.33 to 2.35 bits per pixel. The values below are approximate readings from the plotted figures; the experiment CSV should be treated as the authoritative source for exact numerical results.

Figure 2 shows the average PSNR as a function of BPP. The plotted PSNR increases from approximately 26.7 dB at the lowest bitrate to approximately 38 dB at the highest bitrate. This illustrates the expected rate–distortion trade-off: allocating more bits generally improves reconstruction quality.

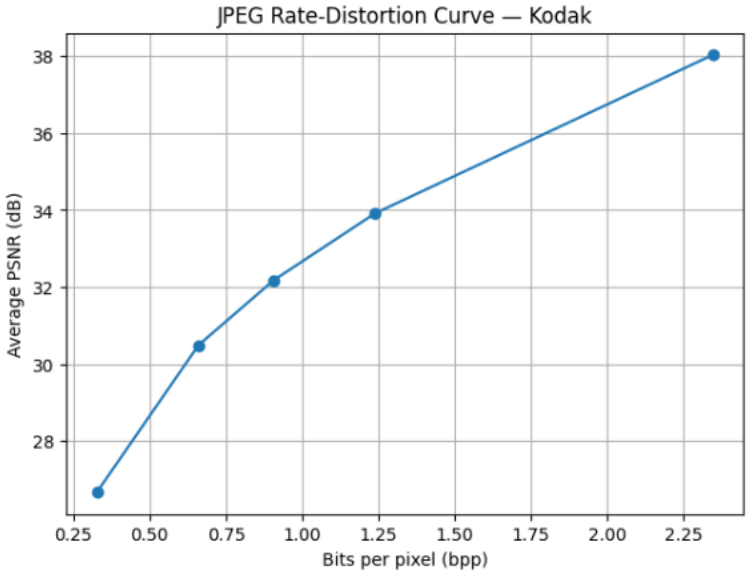


Figure 2: JPEG rate–distortion curve using average PSNR on the Kodak dataset.

Figure 3 shows the average MS-SSIM at the same plotted operating points. The value increases from approximately 0.894 to approximately 0.993 as BPP increases. The curve also suggests that the improvement in MS-SSIM becomes smaller at higher bitrates, which is consistent with diminishing perceptual-quality gains as more bits are allocated.

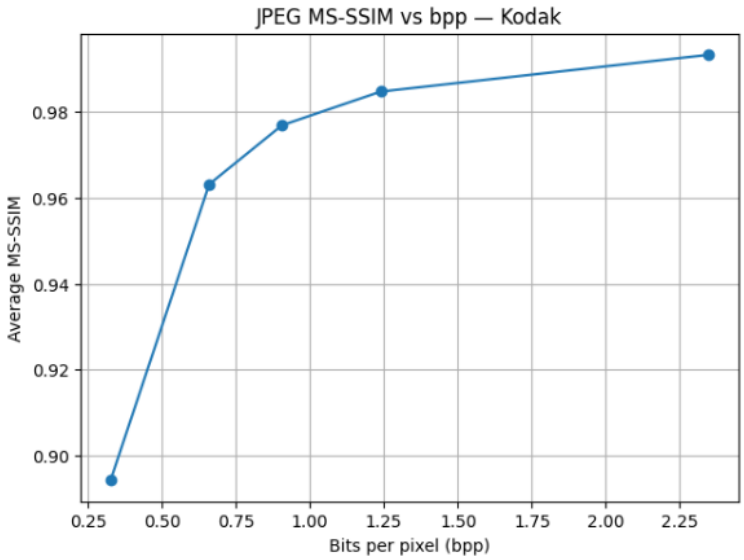


Figure 3: Average JPEG MS-SSIM versus BPP on the Kodak dataset.

These results are preliminary baseline measurements, not a comparison against the learned codec. The learned-codec results will be added after the trained model and its actual bitstream encoder/decoder are available.

5.2 Evaluation Implementation

The evaluation pipeline is implemented in Python. The main libraries used are:

- **Pillow (PIL):** JPEG compression and image decoding.
- **NumPy:** Image-array processing and numerical operations.
- **Pandas:** Organizing and exporting per-image and averaged experimental results to CSV.
- **Matplotlib:** Generating the rate–distortion plots.
- **scikit-image:** PSNR calculation.
- **pytorch-msssim:** MS-SSIM calculation.
- **PyTorch:** Tensor handling required by the MS-SSIM implementation.

The pipeline processes the Kodak images at each JPEG quality setting, saves the compressed outputs, records their file sizes, decodes them, and calculates the quality metrics against the corresponding original images. The measurements are saved in CSV format so that plots and summary tables can be regenerated.

The metric-calculation functions are kept separate from the codec implementation. This allows the same evaluation functions to be reused for the learned compression model and helps maintain a consistent comparison between JPEG and the learned codec.

6 Feasibility and Compute Plan

The model will be implemented using the CompressAI `bmshj2018-hyperprior` architecture, which corresponds to the scale-hyperprior method. In CompressAI 1.2.8, quality levels 1–5 use $N = 128$ hyperprior channels and $M = 192$ latent channels; levels 6–8 use $N = 192$ and $M = 320$. These are standard model configurations and quality presets. We will use the standard configuration as a reference and test a limited number of reduced-width configurations, with the number of experiments depending on available compute. For each configuration, the rate–distortion setting, including the distortion metric and λ, will be recorded. Reduced-width models will be trained for this project rather than treated as pretrained standard CompressAI models.

We will begin by trying to train the model on a laptop. If the required runtime or memory is not enough, we will use Google Colab when available. The available hardware may vary, so we will record the device, model configuration, image crop size, batch size, and training time for each experiment.

The one-step feasibility check is complete. On an Intel Core i7-11850H laptop with 64 GB RAM and an NVIDIA T1200 Laptop GPU, the CPU-only PyTorch 2.14 and CompressAI 1.2.8 environment loaded the pretrained quality-3 model ($N = 128$, $M = 192$, 5.08 M parameters) and completed one main and auxiliary optimizer step on a 256×256 crop from `DIV2KDataset`. The latent shapes were $y : 192 \times 16 \times 16$ and $z : 128 \times 4 \times 4$; encoder weights changed, and the evaluation-mode loss moved from 2.269 to 2.258. Real compression produced 6,660 bytes (0.813 bpp, including 84 bytes for z); estimated eval bpp was 0.810, within 0.5%. This establishes feasibility of a step, not a training-time estimate or a trained reduced-width model. The automated integration test also completed a training step and compress/decompress round trip using a reduced model ($N = 32$, $M = 48$) on batches from `DIV2KDataset`. All 18 tests passed in about 40 s on CPU. A short training pilot is still needed to estimate step time and peak memory before choosing a full training schedule.

6.1 Minimum Viable Deliverable

The minimum deliverable is one trained reduced-width scale-hyperprior model that encodes and decodes each of the 24 full-resolution Kodak images. We will calculate BPP from the actual encoded bitstream and measure PSNR and MS-SSIM against the original images using the same metric implementations as for JPEG. The results will include per-image measurements, dataset averages, and a rate–distortion comparison at matched BPP. The completed one-step feasibility test establishes that the data-to-model path and real entropy-coded compress/decompress path run; it does not yet satisfy the trained-model portion of this deliverable.

6.2 Stretch Goal

If the minimum experiment is complete by Week 5–6 and compute permits, we will test one additional learned image-compression model and compare it with our reduced-width scale-hyperprior on Kodak. Candidate models will be selected from the approaches in our references: MLIC's multi-reference entropy model (Jiang et al., 2023) or the pruned hyperprior approach ERHP (Luo et al., 2022). Ballé et al. (Ballé et al., 2018) will serve as the scale-hyperprior reference.

7 Risks and Mitigations

- **Student A/B – Training may exceed available compute.** The reduced-width model may still require more memory or training time than the available laptop or Colab session can provide.

The team will begin with a small feasibility run and reduce the batch size, crop size, channel width, or number of training steps if necessary. The selected configuration and any resulting limitations will be documented.

- **Student B/C – The learned codec may not provide a complete bitstream pipeline on schedule.** A model that produces reconstructed tensors but does not yet support actual encoding and decoding cannot be compared fairly using true file-size BPP. The team will prioritize an end-to-end encoder/decoder and record the actual bitstream size. If this remains incomplete, the report will clearly label any estimated rate results and will not present them as measured bitstream BPP.
- **Student C – Inconsistent evaluation may lead to an unfair comparison.** Differences in image dimensions, color conversion, cropping, metric implementation, or data range can change PSNR and MS-SSIM results. To mitigate this risk, the same original Kodak images and consistent preprocessing and metric definitions will be used for JPEG and the learned codec. The evaluation scripts will retain per-image measurements as well as dataset averages, and the source CSV files will be preserved so that plots can be regenerated and checked.
- **Student C – Incorrect bitrate accounting.** Using an estimated entropy value instead of the actual compressed file size could misrepresent the achieved compression rate. For JPEG, BPP will be calculated from the saved JPEG file size. For the learned codec, the calculation will use the complete encoded bitstream, including the data required by the decoder. Any difference between estimated and actual rate will be stated.
- **Student C – Decode-time measurements may be noisy or hardware-dependent.** Runtime can vary with device load, warm-up state, and software configuration. Measurements will therefore be taken on a documented device, after warm-up, over repeated runs. File I/O will be excluded where practical, and the timing method and hardware will be reported. If a consistent timing environment cannot be maintained, decode-time results will be presented as indicative rather than as a definitive hardware comparison.

8 Work Breakdown

9 Student A Deliverables and Current Status

The following data and baseline tasks have been completed:

- Downloaded and verified DIV2K train (800 images), DIV2K validation (100 images) and Kodak (24 images); every image decodes and all are RGB.
- Implemented `DIV2KDataset`: random 256×256 crops with horizontal and vertical flips, returning `float32` RGB tensors in $[0, 1]$ as the agreed interface to the model ($[B, 3, H, W]$).
- Added a shared dataset-location setting (`data/paths.py`) used by the loader, scripts, tests and training notebook.
- Implemented the Pillow JPEG baseline runner and produced the Kodak results in Table 2.
- Connected the one-training-step notebook to `DIV2KDataset` and added an integration test for the data-to-model interface; all 18 automated tests pass.

Table 3: Task allocation by milestone and team member.

Milestone	Student A	Student B	Student C
M1 (Week 3)	Download and verify DIV2K train+val and Kodak; implement `DIV2KDataset` (random 256×256 crops, flips); shared dataset-path configuration; Pillow JPEG baseline runner; data-to-model integration test; hyperprior literature survey.	Review hyperprior design; determine the parameters for the model and run a one-step feasibility test.	Prepare Kodak evaluation dataset; implement BPP, PSNR, and MS-SSIM calculations; establish the JPEG baseline methodology and generate preliminary plots.
M2 (Week 6)	JPEG rate–distortion curve on Kodak in the shared results file; download, clean and inspect Tecnick; help monitor training.	Implement the training and encode/decode pipeline; train the model and verify that it produces a valid bitstream.	Complete and validate JPEG experiments; organize per-image and averaged results; generate and document preliminary rate–distortion curves.
M3 (Week 9)	JPEG on Tecnick and final JPEG/Tecnick comparison table; visual comparison crops; Data and Traditional Baselines report sections.	Finalize the trained model and bitstream encoder/decoder; document the model and training settings and support the final evaluation.	Evaluate the learned codec on Kodak; compare it against JPEG using rate–distortion curves; measure decoding time; prepare final result tables and qualitative comparisons.

- Completed the hyperprior literature survey and drafted the training-data and baseline material in this proposal.

Remaining Student A tasks are to add MS-SSIM values from the shared evaluation functions to the shared results file, prepare Tecnick, and produce visual comparison crops.

10 Student C Deliverables and Current Status

The following evaluation tasks have been completed for the initial JPEG baseline:

- Downloaded and prepared the 24-image Kodak evaluation dataset.
- Implemented BPP calculation using the actual compressed JPEG file size and image dimensions.
- Implemented PSNR and MS-SSIM calculation functions.
- Ran JPEG compression experiments and collected preliminary measurements.
- Generated average PSNR–BPP and MS-SSIM–BPP plots.

The remaining evaluation tasks are to run the same pipeline on the learned codec, verify that BPP is based on the actual encoded bitstream, measure decoding time under a consistent protocol, and produce the final JPEG-versus-learned-codec comparison.

References

Johannes Ballé, David Minnen, Saurabh Singh, Sung Jin Hwang, and Nick Johnston. Variational image compression with a scale hyperprior. In *International Conference on Learning Representations*, 2018.

Gregory K. Wallace. The JPEG still picture compression standard. *IEEE Transactions on Consumer Electronics*, 38(1), 1992.

Johannes Ballé, Valero Laparra, and Eero P. Simoncelli. End-to-end optimized image compression. In *International Conference on Learning Representations*, 2017.

David Minnen, Johannes Ballé, and George Toderici. Joint autoregressive and hierarchical priors for learned image compression. In *Advances in Neural Information Processing Systems*, 2018.

Dailan He, Ziming Yang, Weikun Peng, Rui Ma, Hongwei Qin, and Yan Wang. ELIC: Efficient learned image compression with unevenly grouped space-channel contextual adaptive coding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.

Jinming Liu, Heming Sun, and Jiro Katto. Learned image compression with mixed transformer-CNN architectures. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.

Zhihao Duan, Ming Lu, Zhan Ma, and Fengqing Zhu. Lossy image compression with quantized hierarchical VAEs. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2023.

Jean Bégaint, Fabien Racapé, Simon Feltman, and Akshay Pushparaja. CompressAI: a PyTorch library and evaluation platform for end-to-end compression research. *arXiv preprint arXiv:2011.03029*, 2020.

Eirikur Agustsson and Radu Timofte. NTIRE 2017 challenge on single image super-resolution: Dataset and study. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2017.

Eastman Kodak. Kodak lossless true color image suite. http://r0k.us/graphics/kodak/, n.d.

Nicola Asuni and Andrea Giachetti. TESTIMAGES: a large-scale archive for testing visual devices and basic image processing algorithms. In *Eurographics Italian Chapter Conference (STAG)*, 2014.

Zhou Wang, Eero P. Simoncelli, and Alan C. Bovik. Multiscale structural similarity for image quality assessment. In *Asilomar Conference on Signals, Systems and Computers*, 2003.

Wei Jiang, Jiayu Yang, Yongqi Zhai, Peirong Ning, Feng Gao, and Ronggang Wang. MLIC: Multi-reference entropy model for learned image compression. In *Proceedings of the 31st ACM International Conference on Multimedia*, pages 7618–7627, 2023. doi: 10.1145/3581783.3611694.

Ao Luo, Heming Sun, Jinming Liu, and Jiro Katto. Memory-efficient learned image compression with pruned hyperprior module. In *2022 IEEE International Conference on Image Processing (ICIP)*, 2022. URL https://arxiv.org/abs/2206.10083. Accepted for presentation.